

Exercícios Complementares - Aula 2

Aplicações Avançadas: Densidade de Probabilidade, Autovalores/Autovetores e Mínimos Quadrados

Prof. Eduardo Paiva

Inteligência Artificial Aplicada à Engenharia de Petróleo

CESFI/UDESC Balneário Camboriú



EXERCÍCIO 1: Densidade de Probabilidade

Contexto: Análise da Distribuição de Porosidade em um Reservatório

Um campo petrolífero apresenta porosidade com distribuição normal. Através de dados de 200 poços, foi determinado que:

- Média (μ) = 16%
- Desvio padrão (σ) = 3%

Questões:

- Calcule a probabilidade de um poço ter porosidade entre 12% e 20%
- Qual a probabilidade de encontrar um poço com porosidade superior a 22%?
- Se classificarmos como "alta qualidade" os poços com porosidade no top 10%, qual o valor mínimo de porosidade para essa classificação?

Solução Python:

```
python
```

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# Parâmetros da distribuição
mu = 0.16 # 16%
sigma = 0.03 # 3%

# a)  $P(0.12 < \varphi < 0.20)$ 
prob_a = stats.norm.cdf(0.20, mu, sigma) - stats.norm.cdf(0.12, mu, sigma)
print(f"a)  $P(12\% < \varphi < 20\%) = \{prob\_a:.4f\} = \{prob\_a*100:.2f\}\%$ ")

# b)  $P(\varphi > 0.22)$ 
prob_b = 1 - stats.norm.cdf(0.22, mu, sigma)
print(f"b)  $P(\varphi > 22\%) = \{prob\_b:.4f\} = \{prob\_b*100:.2f\}\%$ ")

# c) Percentil 90% (top 10%)
percentil_90 = stats.norm.ppf(0.90, mu, sigma)
print(f"c) Porosidade mínima para top 10%:  $\{percentil\_90:.3f\} = \{percentil\_90*100:.1f\}\%$ ")

# Visualização
x = np.linspace(0.05, 0.27, 1000)
y = stats.norm.pdf(x, mu, sigma)

plt.figure(figsize=(12, 8))
plt.plot(x, y, 'b-', linewidth=2, label='Densidade de Probabilidade')

# Área entre 12% e 20%
x_fill_a = np.linspace(0.12, 0.20, 100)
y_fill_a = stats.norm.pdf(x_fill_a, mu, sigma)
plt.fill_between(x_fill_a, y_fill_a, alpha=0.3, color='green', label='P(12% <  $\varphi$  < 20%)')

# Área superior a 22%
x_fill_b = np.linspace(0.22, 0.27, 100)
y_fill_b = stats.norm.pdf(x_fill_b, mu, sigma)
plt.fill_between(x_fill_b, y_fill_b, alpha=0.3, color='red', label='P( $\varphi$  > 22%)')

# Linha do percentil 90%
plt.axvline(percentil_90, color='orange', linestyle='--', linewidth=2, label='Top 10%')

plt.xlabel('Porosidade')
plt.ylabel('Densidade de Probabilidade')
plt.title('Distribuição de Porosidade no Reservatório')
plt.legend()

```

```
plt.grid(True, alpha=0.3)
plt.show()
```

Interpretação Geológica:

- **67% dos poços** têm porosidade entre 12-20% (boa para excelente)
- **Apenas 2.3%** têm porosidade > 22% (excepcional)
- **Critério top 10%:** Poços com $\varphi \geq 19.8\%$ são considerados premium

1234 EXERCÍCIO 2: Autovalores e Autovetores (PCA Aplicado)

Contexto: Análise de Componentes Principais em Dados Petrofísicos

Você tem dados de 4 propriedades petrofísicas de 100 poços:

- Porosidade (φ)
- Permeabilidade (k)
- Saturação de água (S_w)
- Resistividade (R)

Objetivo: Reduzir dimensionalidade mantendo 95% da variância

Implementação:

```
python
```

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns

# Gerar dados sintéticos realistas
np.random.seed(42)
n_pocos = 100

# Correlações típicas em reservatórios
#  $\phi$  e  $k$  correlacionados positivamente
#  $S_w$  e  $\phi$  correlacionados negativamente
#  $R$  e  $S_w$  correlacionados positivamente

phi = np.random.normal(0.15, 0.03, n_pocos)
phi = np.clip(phi, 0.05, 0.30)

#  $k$  correlacionado com  $\phi$  (lei de potência)
k = 1000 * (phi**2.5) * np.random.lognormal(0, 0.5, n_pocos)
k = np.clip(k, 1, 2000)

#  $S_w$  inversamente correlacionado com  $\phi$ 
Sw = 0.6 - 0.8*phi + np.random.normal(0, 0.05, n_pocos)
Sw = np.clip(Sw, 0.15, 0.80)

#  $R$  correlacionado com  $S_w$ 
R = 50 * Sw**2 + np.random.normal(0, 5, n_pocos)
R = np.clip(R, 5, 100)

# Criar DataFrame
dados = pd.DataFrame({
    'Porosidade': phi,
    'Permeabilidade': k,
    'Sat_Agua': Sw,
    'Resistividade': R
})

print("Dados Originais:")
print(dados.describe())
print(f"\nMatriz de Correlação:")
correlacao = dados.corr()
print(correlacao)

# Padronizar os dados (essencial para PCA)
```

```

scaler = StandardScaler()
dados_padronizados = scaler.fit_transform(dados)

# Aplicar PCA
pca = PCA()
componentes = pca.fit_transform(dados_padronizados)

# Análise dos autovalores e autovetores
print(f"\n=== ANÁLISE DE AUTOVALORES E AUTOVETORES ===")
print(f"Autovalores (variâncias explicadas):")
for i, valor in enumerate(pca.explained_variance_):
    print(f" PC{i+1}: {valor:.3f} ({pca.explained_variance_ratio_[i]*100:.1f}%)")

print(f"\nVariância acumulada:")
variancia_acumulada = np.cumsum(pca.explained_variance_ratio_)
for i, var_acum in enumerate(variancia_acumulada):
    print(f" PC1-PC{i+1}: {var_acum*100:.1f}%)")

# Autovetores (loadings)
print(f"\nAutovetores (Componentes Principais):")
componentes_df = pd.DataFrame(
    pca.components_.T,
    columns=[f'PC{i+1}' for i in range(len(dados.columns))],
    index=dados.columns
)
print(componentes_df)

# Determinar quantos componentes para 95% da variância
n_componentes_95 = np.argmax(variancia_acumulada >= 0.95) + 1
print(f"\nPara manter 95% da variância: {n_componentes_95} componentes")

# Visualizações
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# 1. Scree Plot
axes[0,0].bar(range(1, len(pca.explained_variance_ratio_)+1),
              pca.explained_variance_ratio_*100)
axes[0,0].set_xlabel('Componente Principal')
axes[0,0].set_ylabel('Variância Explicada (%)')
axes[0,0].set_title('Scree Plot - Variância por Componente')
axes[0,0].axhline(y=5, color='red', linestyle='--', alpha=0.7, label='Limiar 5%')
axes[0,0].legend()

# 2. Variância Acumulada
axes[0,1].plot(range(1, len(variancia_acumulada)+1),
               variancia_acumulada*100, 'bo-')
axes[0,1].axhline(y=95, color='red', linestyle='--', alpha=0.7, label='95%')

```

```

axes[0,1].set_xlabel('Número de Componentes')
axes[0,1].set_ylabel('Variância Acumulada (%)')
axes[0,1].set_title('Variância Explicada Acumulada')
axes[0,1].legend()

# 3. Biplot PC1 vs PC2
axes[1,0].scatter(componentes[:,0], componentes[:,1], alpha=0.6)
axes[1,0].set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
axes[1,0].set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')
axes[1,0].set_title('Biplot: PC1 vs PC2')

# Adicionar vetores das variáveis originais
feature_vectors = pca.components_.T * np.sqrt(pca.explained_variance_)
for i, (variable, vector) in enumerate(zip(dados.columns, feature_vectors)):
    axes[1,0].arrow(0, 0, vector[0]*3, vector[1]*3,
                    head_width=0.1, head_length=0.1,
                    fc=f'C{i}', ec=f'C{i}')
    axes[1,0].text(vector[0]*3.5, vector[1]*3.5, variable,
                   color=f'C{i}', fontweight='bold')

# 4. Heatmap dos loadings
sns.heatmap(componentes_df.iloc[:, :3], annot=True, cmap='coolwarm',
            center=0, ax=axes[1,1])
axes[1,1].set_title('Loadings das Variáveis nos PCs')

plt.tight_layout()
plt.show()

# Aplicar PCA reduzido (95% variância)
pca_reduzido = PCA(n_components=n_componentes_95)
dados_reduzidos = pca_reduzido.fit_transform(dados_padronizados)

print(f"\n=== REDUÇÃO DE DIMENSIONALIDADE ===")
print(f"Dimensões originais: {dados.shape}")
print(f"Dimensões após PCA: {dados_reduzidos.shape}")
print(f"Redução: {(1 - dados_reduzidos.shape[1]/dados.shape[1])*100:.1f}%")
print(f"Variância mantida: {pca_reduzido.explained_variance_ratio_.sum()*100:.1f}%")

```

Interpretação Petrofísica:

- **PC1** geralmente captura a qualidade geral do reservatório
- **PC2** pode representar o contraste entre porosidade e saturação de água
- **Redução dimensional** permite visualização e análise mais eficiente



EXERCÍCIO 3: Método dos Mínimos Quadrados

Contexto: Modelo de Previsão de Produção de Petróleo

Você precisa desenvolver um modelo para prever a produção de petróleo baseado em propriedades do reservatório.

Dados: 50 poços com suas respectivas produções mensais

Implementação Completa:

```
python
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
from scipy import stats

# Gerar dados sintéticos realistas
np.random.seed(123)
n_pocos = 50

# Variáveis independentes (features)
phi = np.random.uniform(0.08, 0.25, n_pocos) # Porosidade (8-25%)
k = np.random.lognormal(3, 1, n_pocos) # Permeabilidade (log-normal)
k = np.clip(k, 10, 1000) # Clip entre 10-1000 mD
h = np.random.uniform(10, 50, n_pocos) # Espessura (10-50 m)
p = np.random.uniform(200, 400, n_pocos) # Pressão (200-400 psi)

# Variável dependente (target) - Produção
# Modelo físico simplificado:  $Q \propto k \cdot h \cdot \phi \cdot p$  + ruído
producao_teorica = 0.1 * k * h * phi * p / 1000 # Fator de escala
ruído = np.random.normal(0, 0.2, n_pocos)
producao = producao_teorica + ruído
producao = np.clip(producao, 0.5, None) # Produção mínima realística

# Criar DataFrame
dados = pd.DataFrame({
    'Porosidade': phi,
    'Permeabilidade': k,
    'Espessura': h,
    'Pressao': p,
    'Producao': producao
})

print("=== DADOS DO PROBLEMA ===")
print(dados.describe())
print(f"\nCorrelação com Produção:")
correlacoes = dados.corr()['Producao'].sort_values(ascending=False)
print(correlacoes)

# Preparar dados para regressão
X = dados[['Porosidade', 'Permeabilidade', 'Espessura', 'Pressao']].values
y = dados['Producao'].values

# Método dos Mínimos Quadrados - Implementação Manual
print(f"\n=== MÉTODO DOS MÍNIMOS QUADRADOS ===")

```

Adicionar coluna de intercepto (β_0)

```
X_com_intercepto = np.column_stack([np.ones(n_pocos), X])
```

Solução analítica: $\beta = (X'X)^{-1}X'y$

```
XtX = X_com_intercepto.T @ X_com_intercepto
```

```
Xty = X_com_intercepto.T @ y
```

```
beta = np.linalg.inv(XtX) @ Xty
```

```
print(f"Coeficientes calculados manualmente:")
```

```
coef_nomes = ['Intercepto', 'Porosidade', 'Permeabilidade', 'Espessura', 'Pressao']
```

```
for nome, coef in zip(coef_nomes, beta):
```

```
    print(f" {nome}: {coef:.6f}")
```

Previsões

```
y_pred_manual = X_com_intercepto @ beta
```

Cálculo do R^2

```
y_mean = np.mean(y)
```

```
SS_tot = np.sum((y - y_mean)**2)
```

```
SS_res = np.sum((y - y_pred_manual)**2)
```

```
r2_manual = 1 - (SS_res / SS_tot)
```

```
print(f"\nR2 (manual): {r2_manual:.4f}")
```

```
print(f"RMSE (manual): {np.sqrt(mean_squared_error(y, y_pred_manual)):.4f}")
```

Comparação com sklearn

```
modelo_sklearn = LinearRegression()
```

```
modelo_sklearn.fit(X, y)
```

```
y_pred_sklearn = modelo_sklearn.predict(X)
```

```
print(f"\n=== COMPARAÇÃO COM SKLEARN ===")
```

```
print(f"Intercepto sklearn: {modelo_sklearn.intercept_:.6f}")
```

```
print(f"Coeficientes sklearn: {modelo_sklearn.coef_}")
```

```
print(f"R2 sklearn: {modelo_sklearn.score(X, y):.4f}")
```

Análise estatística dos resíduos

```
residuos = y - y_pred_manual
```

```
residuos_padronizados = residuos / np.std(residuos)
```

```
print(f"\n=== ANÁLISE DOS RESÍDUOS ===")
```

```
print(f"Média dos resíduos: {np.mean(residuos):.6f}")
```

```
print(f"Desvio padrão: {np.std(residuos):.4f}")
```

Teste de normalidade dos resíduos

```
estatistica, p_valor = stats.shapiro(residuos)
```

```
print(f"Teste de normalidade (Shapiro-Wilk):")
```

```

print(f" Estatística: {estatistica:.4f}")
print(f" P-valor: {p_valor:.4f}")
print(f" Resíduos são normais: {'Sim' if p_valor > 0.05 else 'Não'}")

# Visualizações
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# 1. Valores observados vs preditos
axes[0,0].scatter(y, y_pred_manual, alpha=0.7)
axes[0,0].plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
axes[0,0].set_xlabel('Produção Observada')
axes[0,0].set_ylabel('Produção Predita')
axes[0,0].set_title('Observado vs Predito')
axes[0,0].text(0.05, 0.95, f'R² = {r2_manual:.3f}',
               transform=axes[0,0].transAxes, fontsize=12,
               bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.7))

# 2. Resíduos vs valores preditos
axes[0,1].scatter(y_pred_manual, residuos, alpha=0.7)
axes[0,1].axhline(y=0, color='red', linestyle='--')
axes[0,1].set_xlabel('Valores Preditos')
axes[0,1].set_ylabel('Resíduos')
axes[0,1].set_title('Resíduos vs Preditos')

# 3. Q-Q plot dos resíduos
stats.probplot(residuos, dist="norm", plot=axes[0,2])
axes[0,2].set_title('Q-Q Plot dos Resíduos')

# 4. Histograma dos resíduos
axes[1,0].hist(residuos, bins=15, alpha=0.7, density=True, edgecolor='black')
x_norm = np.linspace(residuos.min(), residuos.max(), 100)
y_norm = stats.norm.pdf(x_norm, np.mean(residuos), np.std(residuos))
axes[1,0].plot(x_norm, y_norm, 'r-', linewidth=2, label='Normal teórica')
axes[1,0].set_xlabel('Resíduos')
axes[1,0].set_ylabel('Densidade')
axes[1,0].set_title('Distribuição dos Resíduos')
axes[1,0].legend()

# 5. Importância das variáveis
importancia = np.abs(beta[1:]) # Excluir intercepto
vars_ordenadas = np.argsort(importancia)[::-1]
axes[1,1].bar(range(len(importancia)), importancia[vars_ordenadas])
axes[1,1].set_xticks(range(len(importancia)))
axes[1,1].set_xticklabels([coef_nomes[i+1] for i in vars_ordenadas], rotation=45)
axes[1,1].set_ylabel('|Coeficiente|')
axes[1,1].set_title('Importância das Variáveis')

```

6. Matriz de correlação

```
corr_matrix = dados.corr()
im = axes[1,2].imshow(corr_matrix, cmap='coolwarm', aspect='auto', vmin=-1, vmax=1)
axes[1,2].set_xticks(range(len(corr_matrix.columns)))
axes[1,2].set_yticks(range(len(corr_matrix.columns)))
axes[1,2].set_xticklabels(corr_matrix.columns, rotation=45)
axes[1,2].set_yticklabels(corr_matrix.columns)
axes[1,2].set_title('Matriz de Correlação')
```

Adicionar valores na matriz

```
for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        axes[1,2].text(j, i, f'{corr_matrix.iloc[i,j]:.2f}',
                        ha='center', va='center',
                        color='white' if abs(corr_matrix.iloc[i,j]) > 0.5 else 'black')
```

```
plt.colorbar(im, ax=axes[1,2], shrink=0.8)
plt.tight_layout()
plt.show()
```

Equação final do modelo

```
print(f"\n=== EQUAÇÃO DO MODELO ===")
print(f"Produção = {beta[0]:.3f} + {beta[1]:.3f}×Porosidade + {beta[2]:.6f}×Permeabilidade + {beta[3]:.3f}×Espessura + {b
```

Exemplo de previsão para um novo poço

```
print(f"\n=== EXEMPLO DE PREVISÃO ===")
novo_poco = np.array([0.18, 150, 25, 300]) #  $\phi=18\%$ ,  $k=150\text{mD}$ ,  $h=25\text{m}$ ,  $p=300\text{psi}$ 
novo_poco_com_intercepto = np.concatenate([[1], novo_poco])
previsao = novo_poco_com_intercepto @ beta
print(f"Novo poço:  $\phi=18\%$ ,  $k=150\text{mD}$ ,  $h=25\text{m}$ ,  $p=300\text{psi}$ ")
print(f"Produção prevista: {previsao:.2f} barris/dia")
```

Intervalo de confiança (aproximado)

```
mse = mean_squared_error(y, y_pred_manual)
std_erro = np.sqrt(mse)
ic_95 = 1.96 * std_erro
print(f"Intervalo de confiança 95%: [{previsao-ic_95:.2f}, {previsao+ic_95:.2f}] barris/dia")
```

Interpretação dos Resultados:

1. **Coefficientes:** Indicam o impacto de cada variável na produção
2. **R^2 :** Percentual da variabilidade explicada pelo modelo
3. **Resíduos:** Verificação das suposições do modelo linear
4. **Significância:** Importância relativa de cada parâmetro

EXERCÍCIOS PROPOSTOS PARA OS ALUNOS

Exercício A: Densidade de Probabilidade

Uma formação geológica apresenta permeabilidade seguindo distribuição log-normal com $\ln(k) \sim N(4.5, 1.2)$.

1. Calcule a mediana da permeabilidade
2. Qual a probabilidade de $k > 200$ mD?
3. Determine o percentil 75%

Exercício B: PCA

Aplique PCA aos dados de porosidade, permeabilidade e saturação de água de 30 poços.

1. Quantos componentes explicam 90% da variância?
2. Interprete o primeiro componente principal
3. Identifique poços outliers

Exercício C: Mínimos Quadrados

Desenvolva um modelo para prever a recuperação de óleo baseado em:

- Porosidade, permeabilidade, viscosidade do óleo
1. Implemente o método analítico
 2. Analise a qualidade dos resíduos
 3. Faça previsões com intervalos de confiança

Bibliografia Complementar

- **Montgomery, D.C.** - Introduction to Statistical Quality Control
- **Strang, G.** - Linear Algebra and Its Applications
- **Wackerly, D.** - Mathematical Statistics with Applications
- **James, G. et al.** - An Introduction to Statistical Learning
- **Jolliffe, I.T.** - Principal Component Analysis

Código Completo para Download

Todos os exercícios acima estão disponíveis como notebooks individuais no Google Colab. Cada exercício inclui:

- Dados sintéticos realísticos

- Implementação passo-a-passo
- Visualizações interpretativas
- Análise dos resultados
- Exercícios propostos para prática

Acesse: [Link para notebooks no Colab - a ser fornecido]